DG INTERNS HUB — TECHNICAL NOTES

TOPIC 04: INTRODUCTION TO SEARCH PROCESSING LANGUAGE (SPL)

Submitted By: Gunjan
Internship: DG Interns Hub Cybersecurity
Submission Date: 31 October 2026
Department: Cybersecurity / SOC
Week: Week 4
Document ID: 04_Introduction_to_SPL.pdf

TABLE OF CONTENTS

1. SPL Pipeline Concept & Syntax
2. Core Search Commands
3. Key Fields & Filtering Mechanisms
4. Advanced Aggregations (`stats`, `eval`, `where`)
5. Practical SOC Query Example & Triage
6. SPL Optimization Best Practices

1. SPL Syntax & Pipeline Concept

Splunk Search Processing Language (SPL) uses the pipe character (|) to feed the output of one command as the input into the next. SPL queries begin with a search term or index filter and progressively refine, transform, or visualize data.

2. Core SPL Commands Reference

- `search`: Filters raw log streams based on key-value pairs or strings.
- `stats`: Calculates summary statistics (e.g., `count`, `dc(dest)`, `sum(bytes)`).
- `eval`: Calculates new fields or evaluates conditional expressions dynamically.
- `table`: Formats results into a clean tabular layout with designated columns.
- `where`: Filters results post-aggregation using mathematical or Boolean evaluations.

3. Practical SOC Example: Brute Force Query

> **Scenario: Detecting High-Frequency Failed Logins via SPL**
>
> **Query Objective:** Identify external IPs with over 15 failed RDP logon attempts in a 5-minute window.

```
index=winlogbeat EventCode=4625 Logon_Type=10
| stats count dc(TargetUserName) as unique_users by Source_Network_Address
| where count > 15
| rename Source_Network_Address as src_ip
| table src_ip, count, unique_users
| sort - count
```

Query Breakdown:

- `index=winlogbeat EventCode=4625` fetches Windows failed logon events.
- `Logon_Type=10` narrows focus to Remote Desktop (RDP) sessions.
- `stats count dc(TargetUserName)...` aggregates total failures and distinct targeted users per source IP.
- `where count > 15` eliminates minor user mistakes, isolating real brute-force patterns.

4. SPL Best Practices

Always restrict searches by specifying explicit `index=` and `sourcetype=`. Limit the time picker window instead of searching "All Time" to prevent memory exhaustion on Indexers.